1. Check if two strings are Anagram

```
from collections import Counter

str1 = "adam"
str2 = "daa"
# In Python, sorted(str) converts the string into a list of characters and sorts them.
if Counter(str1) == Counter(str2):
    print("Hey these is Anagram")
else:
    print("Oops not a Anagram")
```

Output: Oops not a Anagram

- **Algorithm Using Hash Maps**

  1. **Initialize a frequency dictionary** (hash map) for each string.
     - Keys → characters
     - Values → counts of occurrences
  2. Traverse each string and update the frequency counts.
  3. Compare the two dictionaries:
     - If they are identical → strings are anagrams.
     - Otherwise → not anagrams.

- **Time Complexity**

  - Building frequency map for str1 → $O(n)$
  - Building frequency map for str2 → $O(m)$
  - Comparing two hash maps → $O(n+m)$

Total Time Complexity:

$$O(n+m)$$

If both strings are of equal length n: (Linear)

$$O(n)$$

- **Space Complexity**

  - Hash maps store character counts.
  - For English alphabet (26 letters), space is $O(1)$(constant), since the maximum number of unique characters is bounded.
  - For general Unicode strings, space is $O(k)$where k= number of distinct characters.

2. Count frequency of each word

```
# Count frequency of each word
text = "Python is easy, but remembering all function, Data structure is hard"
words = text.split() # breaks the sentence into a list of words
freq_count = {} # It store each word as a key & how many times it appears as the value
for word in words:
    freq_count[word] = freq_count.get(word, 0) + 1
# If the word is already in the dictionary, get its current count. If not, start from 0.

print(freq_count)
```

Output: {'Python': 1, 'is': 2, 'easy,': 1, 'but': 1, 'remembering': 1, 'all': 1, 'function,': 1, 'Data': 1, 'structure': 1, 'hard': 1}

- **Algorithm**

  1. Take an input string (sentence or paragraph).
  2. Split the string into words (using .split() in Python).
  3. Use a hash map (dictionary) to store:
     - Key → word
     - Value → frequency (number of times the word appears)
  4. Traverse through the list of words and update the dictionary & Print the dictionary.

- **Complexity Analysis**

  - **Time Complexity:**
    - Splitting text → $O(n)$, where n= number of characters.
    - Counting words → $O(m)$, where m= number of words.
    - Overall → $O(n+m)$, which simplifies to $O(n)$.
  - **Space Complexity:**
    - Hash map stores unique words.
    - If there are $k$unique words, space = $O(k)$.

3. Find the second largest number in a list

```
# Find the second largest number in a list
numbers = [10, 20, 4, 45, 99, 99]
unique_numbers = list(set(numbers)) # Remove duplicates
unique_numbers.sort(reverse=True) # Sort in descending order

print(f"Second largest number is: {unique_numbers[1]}")
```

Output: Second largest number is: 45

- **Data Structures Used**
  - **List (**numbers**):** Stores the input numbers.
  - **Set (**set(numbers)**):** Removes duplicates. A set is an unordered collection of unique elements.
  - **List again (**list(set(numbers))**):** Converts the set back into a list so we can sort it.
  - **Sorted List (**unique_numbers.sort(reverse=True)**):** Stores numbers in descending order.
- **Time Complexity**
  - **Convert list to set:** $O(n)$(each element inserted once).
  - **Convert set back to list:** $O(n)$.
  - **Sort list:** $O(n\log\ n)$.
  - **Indexing (**unique_numbers[1]**):** $O(1)$.

Total Time Complexity: $O(n\log\ n)$

- **Space Complexity**
  - **Set:** Stores up to $n$unique elements → $O(n)$.
  - **List:** Stores up to $n$elements → $O(n)$.
  - No extra large structures beyond these.

Total Space Complexity: $O(n)$

4. Reverse words in sentence

```
# Reverse words in sentence
text = "Python is kind of english"
result = " ".join(reversed(text.split()))
print(result)
```

Output: english of kind is Python

- **Complexity Analysis**
  - **Time Complexity:**
    - Splitting → $O(n)$
    - Reversing (iterator) → $O(1)$overhead
    - Joining → $O(n)$
    - Overall → $O(n)$
  - **Space Complexity:**
    - Only stores the list of words once → $O(m)$, where $m$is the number of words.
    - No extra reversed copy, so it's slightly more space-efficient than slicing.